IUPC Master Reference Sheet

STL, number theory, data structures, graphs, DP, geometry — printable, 2-column format.

IUPC MASTER REFERENCE (10-page budget)

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
#define vi vector<int>
#define vll vector<ll>
#define pii pair<int,int>
#define all(x) (x).begin(), (x).end()
const ll MOD = 1e9 + 7;
```

STL: VECTOR

```
// sort(v.begin(), v.end());
// sort(v.rbegin(), v.rend());
// sort(v.begin(), v.end(), greater<int>());
// reverse(v.begin(), v.end());
// v.erase(unique(v.begin(), v.end()), v.end());
// int mx = *max_element(v.begin(), v.end());
// ll s = accumulate(v.begin(), v.end(), 0LL);
// auto it = lower_bound(v.begin(), v.end(), x);
// auto it = upper_bound(v.begin(), v.end(), x);
// bool ok = binary_search(v.begin(), v.end(), x);
// next_permutation(v.begin(), v.end());
// v.erase(v.begin() + i);
```

STL: SET / MULTISET / MAP

```
// set<int> s; s.insert(x); s.erase(x); s.count(x);
// s.lower_bound(x); s.upper_bound(x);
// multiset<int> ms; ms.erase(ms.find(x));
// map<int,int> m; m[x]++;
// unordered_map<int,int> um;
// for (auto& [k,v] : m) { ... }
```

STL: HEAP / STACK / QUEUE / DEQUE

```
// priority_queue<int> pq;
// priority_queue<int, vector<int>, greater<int>> pq;
// stack<int> st; queue<int> q; deque<int> dq;
// dq.push_front(x); dq.pop_front();
```

STL: STRING / BIT TRICKS

```
// stoi(s), to_string(x), s.substr(pos,len), s.find("x")
// istringstream iss(s); string tok; while (iss >> tok) { ... }  // split
// __builtin_popcount(x), __builtin_ctz(x), __builtin_clz(x)
// x & (x-1)  -> remove lowest set bit ; x & (-x) -> isolate lowest set bit
```

NUMBER THEORY: BASICS

```
ll gcdF(ll a, ll b) { return b ? gcdF(b, a % b) : a; }
ll lcmF(ll a, ll b) { return a / gcdF(a, b) * b; }
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1; ll g = extgcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1; return g;
}
ll power(ll a, ll b, ll m = MOD) {
    a %= m; ll res = 1;
    while (b > 0) { if (b & 1) res = res * a % m;
    return res;
}
ll modinv(ll a, ll m = MOD) { return power(a, m -
```

NUMBER THEORY: SIEVES & FACTORIZATION

```
vector<bool> sieve(int n) {
    vector<bool> isPrime(n + 1, true); isPrime[0]
    for (int i = 2; (ll)i * i <= n; i++)
        if (isPrime[i]) for (int j = i * i; j <=
    return isPrime;
}
vi spf(int n) {
    vi s(n + 1); iota(all(s), 0);
    for (int i = 2; (ll)i * i <= n; i++)
        if (s[i] == i) for (int j = i * i; j <= n
    return s;
}    // descending
map<ll,int> factorize(ll n) {
    map<ll,int> f;
    for (ll d = 2; d * d <= n; d++) while (n % d
    if (n > 1) f[n]++;
    return f;
}
ll phi(ll n) {
    ll result = n;
    for (ll p = 2; p * p <= n; p++)
        if (n % p == 0) { while (n % p == 0) n /=
    if (n > 1) result -= result / n;
    return result;
}
```

NUMBER THEORY: COMBINATORICS & CRT

```
const int NMAX = 2e5 + 5;
ll fact_[NMAX], invfact_[NMAX];
void precomputeFactorials() {
    fact_[0] = 1;
    for (int i = 1; i < NMAX; i++) fact_[i] = fac
    invfact_[NMAX-1] = modinv(fact_[NMAX-1]);
    for (int i = NMAX-2; i >= 0; i--) invfact_[i]
}
ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact_[n] * invfact_[r] % MOD * invfact
}
ll crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y; ll g = extgcd(m1, m2, x, y);
    if ((r2 - r1) % g != 0) return -1;
    ll L = m1 / g * m2;
    ll res = r1 + m1 * ((x * ((r2 - r1) / g)) % (
    return ((res % L) + L) % L;
}
```

DSU (UNION-FIND)

```
struct DSU {
    vi par, rnk;
    DSU(int n) : par(n), rnk(n, 0) { iota(all(par
    int find(int x) { return par[x] == x ? x : pa
    bool unite(int x, int y) {
        x = find(x); y = find(y); if (x == y) ret
```

```
        if (rnk[x] < rnk[y]) swap(x, y);
        par[y] = x; if (rnk[x] == rnk[y]) rnk[x]++;
        return true;
    }
};
```

FENWICK TREE (BIT)

```
struct Fenwick {
    int n; vll tree;
    Fenwick(int n) : n(n), tree(n + 1, 0) {}
    void update(int i, ll delta) { for (++i; i <= n; i += i & -i) tree[i] += delta; }
    ll query(int i) { ll r = 0; for (++i; i > 0; i -= i & -i) r += tree[i]; return r; }
    ll rangeQuery(int l, int r) { return query(r) - (l ? query(l - 1) : 0); }
};
```

SEGMENT TREE (sum, point update)

```
struct SegTree {
    int n; vll tree;
    SegTree(int n) : n(n), tree(4 * n, 0) {}
    void build(vll& a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int mid = (l + r) / 2;
        build(a, 2*node, l, mid); build(a, 2*node+1, mid+1, r);
        tree[node] = tree[2*node] + tree[2*node+1];
    }
    void update(int node, int l, int r, int idx, ll val) {
        if (l == r) { tree[node] = val; return; }
        int mid = (l + r) / 2;
        if (idx <= mid) update(2*node, l, mid, idx, val); else update(2*node+1, mid+1, r, idx, val);
        tree[node] = tree[2*node] + tree[2*node+1];
    }
    ll query(int node, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return query(2*node, l, mid, ql, qr) + query(2*node+1, mid+1, r, ql, qr);
    }
};
```

SEGMENT TREE (range update, range sum, lazy propagation)

```
struct LazySegTree {
    int n; vll tree, lazy;
    LazySegTree(int n) : n(n), tree(4*n, 0), lazy(4*n, 0) {}
    void push(int node, int l, int r) {
        if (lazy[node] == 0) return;
        tree[node] += lazy[node] * (r - l + 1);
        if (l != r) { lazy[2*node] += lazy[node]; lazy[2*node+1] += lazy[node]; }
        lazy[node] = 0;
    }
    void update(int node, int l, int r, int ql, int qr, ll val) {
        push(node, l, r);
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { lazy[node] += val; push(node, l, r); return; }
        int mid = (l + r) / 2;
        update(2*node, l, mid, ql, qr, val); update(2*node+1, mid+1, r, ql, qr, val);
        tree[node] = tree[2*node] + tree[2*node+1];
    }
    ll query(int node, int l, int r, int ql, int qr) {
        push(node, l, r);
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[node];
        int mid = (l + r) / 2;
        return query(2*node, l, mid, ql, qr) + qu
    }
};
```

SPARSE TABLE (range min)

```
struct SparseTable {
    vector<vi> table; vi logs;
    SparseTable(vi& a) {
        int n = a.size(); int LOG = 32 - __builti
        table.assign(LOG, vi(n)); logs.assign(n+1
        for (int i = 2; i <= n; i++) logs[i] = lo
        table[0] = a;
        for (int k = 1; k < LOG; k++)
            for (int i = 0; i + (1<<k) <= n; i++)
                table[k][i] = min(table[k-1][i],
    }
    int query(int l, int r) { int k = logs[r-l+1]
};
```

TRIE (binary / string)

```
struct Trie {
    struct Node { int children[26] = {0}; bool is
    vector<Node> nodes;
    Trie() { nodes.emplace_back(); }
    void insert(const string& s) {
        int cur = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (!nodes[cur].children[idx]) { node
            cur = nodes[cur].children[idx];
        }
        nodes[cur].isEnd = true;
    }
    bool search(const string& s) {
        int cur = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (!nodes[cur].children[idx]) return
            cur = nodes[cur].children[idx];
        }
        return nodes[cur].isEnd;
    }
};
```

GRAPHS: BFS / DFS

```
vi bfs(int src, vector<vi>& adj) {
    vi dist(adj.size(), -1); queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (dist[v] == -1) {
    }
    return dist;
}
void dfs(int u, vector<vi>& adj, vector<bool>& vi
    vis[u] = true;
```

```
    for (int v : adj[u]) if (!vis[v]) dfs(v, adj);
}
```

GRAPHS: TOPOLOGICAL SORT (Kahn's)

```
vi topoSort(int n, vector<vi>& adj) {
    vi indeg(n, 0);
    for (int u = 0; u < n; u++) for (int v : adj[u]) indeg[v]++;
    queue<int> q; for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    vi order;
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order; // size < n => cycle exists
}
```

GRAPHS: DIJKSTRA

```
vll dijkstra(int src, vector<vector<pii>>& adj) {
    int n = adj.size(); vll dist(n, LLONG_MAX);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({d
    }
    return dist;
}
```

GRAPHS: MST (Kruskal, needs DSU)

```
struct Edge { int u, v; ll w; };
ll kruskalMST(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](Edge& a, Edge& b){ return a.w < b.w; });
    DSU dsu(n); ll total = 0;
    for (auto& e : edges) if (dsu.unite(e.u, e.v)) total += e.w;
    return total;
}
```

GRAPHS: LCA (binary lifting)

```
struct LCA {
    int LOG; vector<vi> up; vi depth; vector<vi>& adj;
    LCA(int n, vector<vi>& adj_) : adj(adj_) {
        LOG = 32 - __builtin_clz(max(n,1)) + 1;
        up.assign(LOG, vi(n, 0)); depth.assign(n, 0);
    }
    void dfs(int u, int p) {
        up[0][u] = p;
        for (int k = 1; k < LOG; k++) up[k][u] = up[k-1][up[k-1][u]];
        for (int v : adj[u]) if (v != p) { depth[v] = depth[u]+1; dfs(v, u); }
    }
    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        int diff = depth[u] - depth[v];
        for (int k = 0; k < LOG; k++) if (diff & (1<<k)) u = up[k][u];
        if (u == v) return u;
        for (int k = LOG-1; k >= 0; k--) if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v];
        return up[0][u];
    }
};
```

BINARY SEARCH / TWO POINTERS / KADANE

```
ll binSearch(ll lo, ll hi, function<bool(ll)> pre
    while (lo < hi) { ll mid = lo + (hi-lo)/2; if
    return lo;
}
ll kadane(vll& a) {
    ll best = a[0], cur = a[0];
    for (int i = 1; i < (int)a.size(); i++) { cur
    return best;
}
```

PREFIX SUM (1D / 2D) & COORDINATE COMPRESSION

```
vll prefixSum(vll& a) {
    vll pre(a.size()+1, 0);
    for (int i = 0; i < (int)a.size(); i++) pre[i
    return pre;
}
vi compress(vi a) { // returns rank (0-indexed) o
    vi sorted_ = a;
    sort(all(sorted_)); sorted_.erase(unique(all(
    for (auto& x : a) x = lower_bound(all(sorted_
    return a;
}
```

STRING: KMP & HASHING

```
vi computeLPS(string& pat) {
    int n = pat.size(); vi lps(n, 0); int len = 0
    while (i < n) {
        if (pat[i] == pat[len]) lps[i++] = ++len;
        else if (len) len = lps[len-1];
        else lps[i++] = 0;
    }
    return lps;
}
vi kmpSearch(string& text, string& pat) {
    vi lps = computeLPS(pat), result; int i=0, j=
    while (i < (int)text.size()) {
        if (text[i]==pat[j]) { i++; j++; }
        if (j==(int)pat.size()) { result.push_bac
        else if (i<(int)text.size() && text[i]!=p
    }
    return result;
}
struct StringHash {
    vll h, p; ll base=131, mod=1e9+7;
    StringHash(string& s) {
        int n=s.size(); h.assign(n+1,0); p.assign
        for (int i=0;i<n;i++) { h[i+1]=(h[i]*base
    }
    ll get(int l, int r) { return ((h[r+1]-h[l]*p
};
```

DP: LIS, KNAPSACK, BITMASK, DIGIT DP

```
int LIS(vi& a) {
    vi tails;
    for (int x : a) { auto it = lower_bound(all(t
    return tails.size();
}
```

```
int knapsack01(vi& wt, vi& val, int cap) {
    vi dp(cap+1, 0);
    for (int i = 0; i < (int)wt.size(); i++)
        for (int w = cap; w >= wt[i]; w--) dp[w] = max(dp[w], dp[w-wt[i]] + val[i]);
    return dp[cap];
}
// Bitmask DP skeleton (Traveling Salesman style): dp[mask][i] = min cost visiting set 'mask' ending at i
// for (int mask = 0; mask < (1<<n); mask++)
//   for (int i = 0; i < n; i++) if (mask & (1<<i))
//     for (int j = 0; j < n; j++) if (!(mask & (1<<j)))
//       dp[mask|(1<<j)][j] = min(dp[mask|(1<<j)][j], dp[mask][i] + cost[i][j]);

// Digit DP skeleton: count numbers <= N satisfying a digit property
// ll dp[20][state][2]; memo on (position, custom-state, tight-bound-flag)
// solve(pos, state, tight): if pos==len return valid(state);
//   for d in 0..(tight? digits[pos] : 9): recurse(pos+1, newState, tight && d==digits[pos]);
```

MATRIX EXPONENTIATION

```
typedef vector<vll> Matrix;
Matrix matMul(Matrix& A, Matrix& B, ll mod = MOD) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Matrix C(n, vll(m, 0));
    for (int i=0;i<n;i++) for (int j=0;j<m;j++) for (int x=0;x<k;x++)
        C[i][j] = (C[i][j] + A[i][x]*B[x][j]) % mod;
    return C;
}
Matrix matPow(Matrix A, ll p, ll mod = MOD) {
    int n = A.size();
    Matrix res(n, vll(n, 0));
    for (int i=0;i<n;i++) res[i][i] = 1;
    while (p > 0) { if (p & 1) res = matMul(res, A, mod); A = matMul(A, A, mod); p >>= 1; }
    return res;
}
```

GEOMETRY: BASICS

```
struct Pt { ll x, y; };
ll cross(Pt O, Pt A, Pt B) { return (A.x-O.x)*(B.y-O.y) - (A.y-O.y)*(B.x-O.x); }
ll dist2(Pt A, Pt B) { return (A.x-B.x)*(A.x-B.x) + (A.y-B.y)*(A.y-B.y); }
// orientation: >0 counter-clockwise, <0 clockwise, ==0 collinear
// convex hull (Andrew's monotone chain): sort points, build lower+upper hull using cross()
```

GRAPHS: BELLMAN-FORD (handles negative weights)

```
bool bellmanFord(int n, vector<Edge>& edges, int src, vll& dist) {
    dist.assign(n, LLONG_MAX); dist[src] = 0;
    for (int i = 0; i < n - 1; i++)
        for (auto& e : edges)
            if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v])
                dist[e.v] = dist[e.u] + e.w;
    for (auto& e : edges) // check for negative cycle
        if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v]) return false;
    return true;
}
```

GRAPHS: SCC (Kosaraju's algorithm)

```
void fillOrder(int u, vector<vi>& adj, vector<bool>& vis, vector<int>& order) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) fillOrder(v, adj, vis, order);
    order.push_back(u);
}
void dfsCollect(int u, vector<vi>& radj, vector<b
    vis[u] = true; comp.push_back(u);
    for (int v : radj[u]) if (!vis[v]) dfsCollect
}
vector<vi> kosarajuSCC(int n, vector<vi>& adj, ve
    vector<bool> vis(n, false); vector<int> orden
    for (int i = 0; i < n; i++) if (!vis[i]) fill
    fill(all(vis), false);
    vector<vi> comps;
    for (int i = n - 1; i >= 0; i--) {
        int u = order[i];
        if (!vis[u]) { vi comp; dfsCollect(u, rad
    }
    return comps; // each inner vector = one stro
}
```

GRAPHS: BRIDGES & ARTICULATION POINTS (Tarjan)

```
struct BridgeFinder {
vector<vi>& adj; int n, timer = 0;
    vi disc, low; vector<bool> vis;
vector<pii> bridges;
BridgeFinder(int n, vector<vi>& adj_) : adj(a
    void dfs(int u, int parent) {
        vis[u] = true; disc[u] = low[u] = timer++
        for (int v : adj[u]) {
            if (v == parent) continue;
            if (vis[v]) low[u] = min(low[u], disc
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > disc[u]) bridges.pus
            }
        }
    }
    void run() { for (int i = 0; i < n; i++) if (
};
```

GEOMETRY: CONVEX HULL (Andrew's monotone chain)

```
vector<Pt> convexHull(vector<Pt> pts) {
    int n = pts.size(), k = 0;
    if (n < 3) return pts;
    sort(pts.begin(), pts.end(), [](Pt&a, Pt&b){
    vector<Pt> hull(2*n);
    for (int i = 0; i < n; i++) { // build lower
        while (k>=2 && cross(hull[k-2],hull[k-1],
        hull[k++] = pts[i];
    }
    for (int i = n-2, l = k+1; i >= 0; i--) { //
        while (k>=l && cross(hull[k-2],hull[k-1],
        hull[k++] = pts[i];
    }
    hull.resize(k-1);
    return hull;
}
```

MO'S ALGORITHM (offline range queries, skeleton)

```
// Sort queries by (block = L / blockSize, then R). Maintain add(x)/remove(x) for current window.
// int blockSize = max(1, (int)sqrt(n));
// sort(queries.begin(), queries.end(), [&](auto&a, auto&b){
//     int ba = a.l/blockSize, bb = b.l/blockSize;
//     if (ba != bb) return ba < bb;
//     return (ba & 1) ? a.r > b.r : a.r < b.r;      // alternate direction to save moves
// });
// int curL=0, curR=-1;
// for (auto& q : queries) {
//     while (curR < q.r) add(++curR);
//     while (curL > q.l) add(--curL);
//     while (curR > q.r) remove(curR--);
//     while (curL < q.l) remove(curL++);
//     ans[q.idx] = currentAnswer;
// }
```